

CSC 212: Data Structures and Abstractions

07: Stacks

Prof. Marco Alvarez

Department of Computer Science and Statistics
University of Rhode Island

Fall 2026



Motivation

- An AI agent asks an LLM for a **tool call**, written in JSON
- The model streams its answer, then hits its output-token limit mid-sentence:

```
{"tool": "search", "args": {"q": "stacks", "filters": [1, 2
```

- Two questions before we can use it:
 - ✓ is it **well-formed**: does every bracket close, in the right order?
 - ✓ if not, which closing brackets would **repair** it?

The agent framework must decide whether the text so far is valid, and may repair it if not. Brackets inside JSON strings do not count.

2

Q7-1

- Which closing sequence makes this text balanced?

```
{"a": [1, {"b": 2}
```

- A. `}]`
- B. `}]}`
- C. `}}}`
- D. `}}}`
- E. None of the above

3

Stacks

Stacks

- A **stack** is a container of objects that are inserted and removed according to the last-in first-out (LIFO) principle
 - ✓ objects can be inserted at any time
 - ✓ only the most recently inserted object can be removed at any time
- The fundamental operations are **push** and **pop**
- Everyday stacks
 - ✓ the back button of a web browser
 - ✓ undo in a text editor



5

The stack ADT

- An **abstract data type** (ADT) specifies what each operation does, not how (*the contract*)
 - ✓ the data structure decides how and at which cost
- The stack ADT supports:

operation	what it does
push(e)	insert element e at the top of the stack
pop()	remove the top element; an error occurs if the stack is empty
top()	return a reference to the top element, without removing it; an error occurs if the stack is empty
size()	return the number of elements in the stack
empty()	return true if the stack is empty, and false otherwise

*No indexing, no search, no access to anything but the top.
In C++ (and in the STL), pop returns nothing: read the element with top, then pop.*

6

Q7-2

- Starting from an empty stack, run
 - ✓ push(5) push(3) pop() push(2) push(8) pop()
pop() push(9) push(1)
- What does the stack hold, from bottom to top?
 - A. 5, 9, 1
 - B. 1, 9, 5
 - C. 5, 2, 9, 1
 - D. 3, 9, 1
 - E. None of the above

7

Implementation

- Using dynamic arrays
 - ✓ push and pop at the end of the array (easier and efficient)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
4	3	-5	0	9	-2	7	1								

top

Push	$\Theta(1)$ amortized
Pop	$\Theta(1)$

push/pop at end

Push	$\Theta(n)$
Pop	$\Theta(n)$

push/pop at front

<https://www.cs.usfca.edu/~galles/visualization/StackArray.html>

8

The Stack class

```
class Stack {
private:
    std::vector<int> m_data; // the top is m_data.back()

public:
    void push(int e) { m_data.push_back(e); }

    void pop() {
        if (m_data.empty())
            throw std::underflow_error("pop of empty stack");
        m_data.pop_back();
    }

    const int& top() const {
        if (m_data.empty())
            throw std::underflow_error("top of empty stack");
        return m_data.back();
    }

    size_t size() const { return m_data.size(); }
    bool empty() const { return m_data.empty(); }
};
```

std::vector owns the memory, so copying a Stack copies the elements: no rule of three needed here.

9

Cost of each operation

operation	fixed-capacity array	dynamic array
push(e)	$\Theta(1)$, error when full	$\Theta(1)$ amortized, $\Theta(n)$ worst case
pop()	$\Theta(1)$	$\Theta(1)$
top()	$\Theta(1)$	$\Theta(1)$
size(), empty()	$\Theta(1)$	$\Theta(1)$
space	$\Theta(n)$, n fixed in advance	$\Theta(n)$

every operation touches only the top: nothing ever moves

10

std::stack

- **std::stack** is a **container adapter**
 - ✓ it wraps another container and exposes only the stack operations
 - ✓ the underlying container is **std::deque** by default

```
#include <iostream>
#include <stack>

int main() {
    std::stack<int> s;
    s.push(10);
    s.push(20);
    s.push(30);
    std::cout << s.top() << ' ' << s.size() << '\n';
    s.pop();
    std::cout << s.top() << '\n';
}
```

11

Q7-3

- What does this print?

- A. "200 300 | 100"
- B. "300 200 | 100"
- C. "100 200 300 |"
- D. "300 200 100 |"
- E. None of the above

```
std::stack<int> s1, s2;

s1.push(100);
s2.push(s1.top());
s1.pop();
s1.push(200);
s1.push(300);
s2.push(s1.top());
s1.pop();
s2.push(s1.top());
s1.pop();

s1.push(s2.top());
s2.pop();
s1.push(s2.top());
s2.pop();

while (!s1.empty()) {
    std::cout << s1.top() << ' ';
    s1.pop();
}

std::cout << " | ";
while (!s2.empty()) {
    std::cout << s2.top() << ' ';
    s2.pop();
}
```

12

Q7-4

▸ What does this program do?

```
#include <iostream>
#include <stack>

int main() {
    std::stack<int> s;
    s.pop();
    std::cout << "after pop, size = " << s.size() << '\n';
}
```

- A. It does not compile: `pop` on an empty stack is a *type error*
- B. It throws `std::underflow_error`
- C. It prints “after pop, size = 0”: popping an empty stack does nothing
- D. *Undefined behavior*: `std::stack` does not check for empty
- E. None of the above

13

Matching brackets

▸ Back to the brackets problem. Scan left to right, one character at a time:

- ✓ opening bracket: *push* it
- ✓ closing bracket: the stack must not be empty, and its top must be the same type; then *pop*
- ✓ at the end: the stack must be *empty*

▸ Trace the algorithm with the following inputs:

- ✓ { [()] }
- ✓ ([])
- ✓ ()
- ✓ ((

14

The code

```
bool is_balanced(const std::string& s) {
    std::stack<char> open;
    for (char c : s) {
        if (c == '{' || c == '[' || c == '(') {
            open.push(c); // remember it
        } else if (c == '}' || c == ']' || c == ')') {
            if (open.empty()) return false; // nothing to close
            if (!matches(open.top(), c)) return false; // wrong type
            open.pop(); // matched
        }
    }
    return open.empty(); // anything left open?
}
```

matches(open, close) is true for the pairs {}, [], {}

15

Q7-5

▸ Which string is balanced?

- A. (D]
- B. (((
- C. ([{()})
- D.)(
- E. None of the above

16

Q7-6

- In the worst case, how much extra space does `is_balanced` use on a string of n characters?

- A. $\Theta(1)$
- B. $\Theta(\log n)$
- C. $\Theta(n)$
- D. $\Theta(n^2)$
- E. None of the above

17

Example application

- Fully parenthesized infix expressions
 - ✓ infix: each operator sits between its two operands
 - ✓ every operator, with its two operands, is wrapped in parentheses
 - ✓ no precedence rules needed: the parentheses fix the order

$((5 + ((10 - 4) * (3 + 2))) + 25)$

18

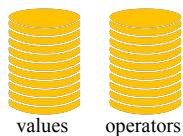
Dijkstra's two-stack algorithm

- Two stacks: values for operands, ops for operators

$(5 + (10 - 4))$

- Read the expression left to right:

- ✓ `(`: ignore it
- ✓ number: push it onto values
- ✓ operator: push it onto ops
- ✓ `)`: pop one operator and two values, apply operator, push the result onto values
- the value popped *first* is the *right* operand



- At the end, the result is the only element of values

algorithm runs
in $\Theta(n)$ time

19

The code

```
int eval(const std::string& exp) {
    std::stack<int> values;
    std::stack<char> ops;
    for (size_t i = 0; i < exp.size(); ++i) {
        char c = exp[i];
        if (std::isdigit(c)) {
            int v = 0;
            while (i < exp.size() && std::isdigit(exp[i]))
                v = 10 * v + (exp[i++] - '0'); // read every digit
            --i;
            values.push(v);
        } else if (c == '+' || c == '-' || c == '*' || c == '/') {
            ops.push(c);
        } else if (c == ')') {
            int right = values.top(); values.pop(); // popped first
            int left = values.top(); values.pop();
            char op = ops.top(); ops.pop();
            values.push(apply(op, left, right));
        } // '(' and spaces: skip
    }
    return values.top();
}
```

20

Q7-7

- Run the two-stack algorithm on:

$$((5 + ((10 - 4) * (3 + 2))) + 25)$$

- what is the largest number of elements the values stack holds at any moment, and what is the result?

- A. 2 elements, result 60
- B. 4 elements, result 60
- C. 3 elements, result 60
- D. 6 elements, result 60
- E. None of the above

21

Programming exercises

- Write a function to reverse a `std::vector<int>` using a stack
 - push every element, then pop them all back into the vector
 - what is the running time? The extra space?
- Write a function to check correct HTML tags: `<body> ... </body>`
 - adapt `is_balanced` to check that the tags in a string match
 - hint: push the tag name when you read an opening tag; on a closing tag, compare names
 - e.g.: `<body><h1>Stacks</h1><p>LIFO <b>only</b></p></body>`

22